

UrbanPulse AI

AI-Powered Urban Traffic Intelligence

From CCTV footage to searchable vehicle trajectories and traffic analytics

PRODUCT REQUIREMENTS DOCUMENT

Consolidated product definition, scope, architecture and build specification — v1.0

Problem Statement ID	SIH26127
Problem Statement Title	AI-Powered Urban Traffic Intelligence
Theme	Smart Automation
PS Category	Software
Event	Smart India Hackathon 2026
Team Name	Trace Forge
Product Name	UrbanPulse AI
Document Version	1.0 — Baseline
Status	Approved for build

This document is the single consolidated source of truth for UrbanPulse AI. It supersedes scattered notes and briefs, and is the reference PRD for all future development beyond the hackathon.

1. Document Control

1.1 Purpose of this document

This Product Requirements Document (PRD) is the consolidated product definition for **UrbanPulse AI**, the solution built by **Team Trace Forge** for Smart India Hackathon 2026, Problem Statement **SIH26127 — AI-Powered Urban Traffic Intelligence**. It merges every artefact the team has produced into one authoritative reference: the SIH Idea presentation, the internal technical briefing, the locked API contract, the AI/ML build specification, the verified technology stack, the module-owner scoping sheet and the team work division.

It serves three audiences at once: **evaluators**, who need the idea, novelty and feasibility in one place; **the build team**, who need requirements, contracts and ownership; and **future maintainers**, who will use this as the baseline PRD when the project continues past the hackathon.

1.2 Source artefacts consolidated into this PRD

Source artefact	Contributed to	Authority in this PRD
SIH 2026 Idea Presentation (Trace Forge)	Vision, problem framing, differentiators, impact	Authoritative for the submitted idea
UrbanPulse AI — Technical Briefing	Architecture layers, identity fusion, mobility graph	Authoritative for architecture
SIH26127 — API Contract (Locked v1.0)	Services, endpoints, payloads, ownership	Authoritative for the MVP build
Final AI/ML Build Specification	Models, features, scoring, evaluation, versions	Authoritative for the intelligence layer
SIH26127 — Final Technology Stack (verified)	Tool selection and MVP scope discipline	Authoritative for stack decisions
Module Owners & Scoping / Team Work Division	Roles, deliverables, interface contracts	Authoritative for ownership
Full ANPR pipeline flow diagram	End-to-end reference figure	Illustrative

1.3 Change control

Freeze rule. Section 12 (API Contract) and Section 11 (Data Model) are **locked specifications**. No field name or type changes without agreement from every owner consuming that endpoint, followed by a version bump of this document. This PRD only has value if the team treats it as locked, not as a suggestion.

Version	Date	Author	Summary
1.0	Baseline	Team Trace Forge	Initial consolidated PRD from all Phase-0 artefacts
1.1	Planned	Module owners	Post-prototype update: measured accuracy, real camera coordinates

1.4 Contents

2	Executive Summary	Vision
3	Problem Statement and Context	Problem

4	Goals, Non-Goals and Success Criteria	Objectives
5	Target Users and Personas	Users
6	Product Scope: MVP, Phase 2 and Future	Scope
7	Solution Overview and Core Differentiators	Solution
8	System Architecture	Architecture
9	Functional Requirements	Requirements
10	Technology Stack (Locked)	Stack
11	Data Model and Database Design	Data
12	API Contract (Locked Spec v1.0)	Interfaces
13	AI / ML Specification	Intelligence
14	Datasets and Data Strategy	Data
15	Non-Functional Requirements	NFRs
16	Privacy, Ethics and Responsible Deployment	Governance
17	Feasibility, Risks and Mitigations	Risk
18	Team, Ownership and Interface Contracts	Team
19	Roadmap and Definition of Done	Delivery
20	Impact and Benefits	Impact
21	Open Questions and Outstanding Items	Open
22	References	Appendix

2. Executive Summary

Indian cities already own thousands of CCTV cameras. Each one observes a single fixed location and stores what it sees. Nothing joins those observations together, so a city with continuous camera coverage still has no continuous understanding of vehicle movement. Investigations are done by manually scrubbing footage feed by feed, and existing ANPR deployments only upgrade that to "which plate was seen at which camera" — a record of sightings, not an understanding of a journey.

UrbanPulse AI converts existing CCTV feeds into structured vehicle movement intelligence. The system detects vehicles and number plates from CCTV using YOLO, tracks each vehicle across frames with ByteTrack, reads and refines the plate with a fine-tuned PaddleOCR model using multi-frame voting, corrects noisy reads against Indian plate grammar and RTO rules, stores every geo-tagged sighting in PostgreSQL + PostGIS, links sightings across cameras into searchable vehicle trajectories, reconstructs the likely route through camera blind spots, flags unusual movement patterns, and presents everything through a GIS command dashboard with live alerts.

Core differentiator. Accuracy is improved at the **journey level**, not only at the individual OCR-frame level. A single frame may misread a plate; a sequence of frames plus cross-camera evidence plus Indian plate constraints will not. This is what converts unreliable per-frame ANPR into reliable vehicle identity.

The engineering contribution is deliberately scoped. Team Trace Forge does not claim to have invented the perception models — YOLO detects, ByteTrack tracks, PaddleOCR reads. **The contribution is the reasoning layer built on top of those outputs:** journey-level identity, a city mobility graph, uncertainty-aware trajectory reconstruction, next-location prediction and anomaly scoring — every output carrying a confidence value so that a human, not the system, makes the decision.

2.1 The product in one line

CCTV video → Detect → Track → Read plate → Correct → Store
→ Connect across cameras → Reconstruct route → Predict → Analyse → Dashboard

2.2 What ships in the prototype

Capability	Prototype behaviour
Automated vehicle search	Search any plate and see every camera it passed, in order, on a map
Journey-level plate identity	Multi-frame OCR voting + Indian plate rules instead of one raw read
Cross-camera trajectory	Geo-tagged sightings linked into an ordered, time-stamped path
Real-time alerting	Blacklist match pushed live to the dashboard over WebSocket
Anomaly flags	Rule-based checks first, Isolation Forest layered on top
Traffic analytics	Heatmap of sighting density, busiest camera, congestion hotspots
Access control	Admin and officer roles, token-authenticated API

3. Problem Statement and Context

3.1 The situation today

A vehicle is detected at Camera A at 10:00. It leaves the field of view. It is seen again at Camera F at 10:20. Between those two moments there are twenty minutes with no observation at all. Which route did it take? Did it pass other cameras? Where might it appear next? Today, nothing in the deployed stack answers these questions. More cameras do not automatically mean more intelligence.

3.2 Five gaps in current approaches

Gap	Name	Description
GAP-01	Cameras stay isolated	Each camera reasons only about its own frame; nothing links the frames of two cameras.
GAP-02	Journeys go missing	The minutes between sightings are simply absent from the record.
GAP-03	ANPR breaks on real conditions	Identity collapses when the plate is dirty, blurred, occluded, stacked on two rows, or shot at a poor angle.
GAP-04	Reactive, not forward-looking	Systems answer where a vehicle <i>was</i> , never where it plausibly goes next.
GAP-05	No movement intelligence	Raw sightings do not explain routes, patterns or meaningful deviation.

3.3 Three generations of camera intelligence

Generation	Question it answers	Where it stops
01 • Manual review	What happened at this camera, at this time?	No team can watch thousands of feeds continuously.
02 • ANPR platforms	Where was this plate last seen?	Identity collapses on a bad read; nothing between two sightings.
03 • UrbanPulse AI	Which sightings are one vehicle, which routes are plausible, where next, how unusual?	Judgement — what a deviation actually means — is left to an authorised human.

3.4 Capability comparison

Capability	Manual review	ANPR platform	UrbanPulse AI
Vehicle identity	The operator's eye	Plate string only	Plate + appearance + feasibility evidence
Cross-camera link	Assembled by hand	Exact plate match	Probabilistic, confidence-scored
Journey between cameras	Guessed	Not represented	Reconstructed and ranked
Where it goes next	Operator instinct	Not supported	Ranked next-camera prediction
Unusual movement	Noticed by chance	Hotlist rules only	Learned deviation scoring
Uncertainty	Undocumented	Match or no match	A confidence value on every output

3.5 How UrbanPulse AI addresses the problem

Manual CCTV search → automated vehicle search

Converts hours of disconnected footage into structured, searchable vehicle records and trajectories.

Noisy ANPR → reliable journey-level identity

Multi-frame voting, cross-camera agreement and Indian plate constraints correct inconsistent OCR reads.

Isolated camera data → connected traffic intelligence

Geo-tagged observations are linked across locations to reveal movement, anomalies and traffic patterns.

4. Goals, Non-Goals and Success Criteria

4.1 Product goals

ID	Goal	What "achieved" looks like
G-01	Turn raw CCTV into structured observations	Every readable vehicle produces a plate, camera, timestamp and location record
G-02	Make plate identity reliable at journey level	Multi-frame voting measurably beats single-frame OCR on the evaluation set
G-03	Make any vehicle searchable city-wide	One query returns the full ordered sighting history of a plate
G-04	Surface threats in real time	A blacklisted plate raises a dashboard alert within seconds of ingestion
G-05	Expose movement patterns, not just points	Heatmap, hotspots and anomaly flags derived from stored sightings
G-06	Keep every output uncertainty-aware	No match, prediction or anomaly is presented without a confidence value
G-07	Run without new hardware	Works on existing camera infrastructure; demo runs on a standard laptop

4.2 Non-goals

- **Not an autonomous enforcement system.** UrbanPulse AI issues no challans, imposes no penalties and takes no automated action against any vehicle or person. It is decision support for an authorised human.
- **Not a claim of inventing perception models.** YOLO, ByteTrack and PaddleOCR are existing open-source tools used as components.
- **Not a face-recognition or person-identification system.** The unit of analysis is a vehicle, never an individual.
- **Not a real-time city-scale deployment in the prototype.** The prototype targets one zone and a small camera set; scale-out is an architecture property, not a hackathon deliverable.
- **No claim of "flawless in all conditions".** Accuracy depends on camera coverage, lighting and data quality, and is reported honestly.

4.3 Success criteria

Dimension	Metric	Prototype target
Plate detection	Detection rate on test footage	Detection is the solved stage; maintain current fine-tuned performance

Plate recognition	Exact-string accuracy, single frame vs multi-frame voting	Voting must show a measurable improvement over single-frame reads
Journey identity	Correct cross-camera linkage rate on the demo set	Correct on the scripted three-camera scenario
Next-location prediction	Top-1, Top-3, Top-5 accuracy, MRR, log loss, calibration error	Report measured values; no blind accuracy promises
Anomaly detection	Precision / recall against reviewed flags	Every flag carries a human-readable reason
Latency	Ingestion to dashboard alert	Alert visible within seconds of the sighting
Reproducibility	Cold start on a clean machine	One <code>docker compose up</code> brings the full stack live

Evaluation honesty rule. Accuracy targets are not promised in the abstract. Every number reported in the final demo must come from a measured run on a chronologically split evaluation set, and the conditions under which it was measured must be stated alongside it.

5. Target Users and Personas

User	What they need	What UrbanPulse AI gives them
Law enforcement agencies Primary	To locate a stolen or wanted vehicle quickly across a city, with evidence they can justify	City-wide plate search, ordered trajectory history, real-time alerts and a confidence value behind every suggestion
Traffic authorities Primary	Live visibility into congestion zones, movement density and corridor load	Heatmaps, vehicle counts, busiest-camera ranking and time-based movement trends
Smart city command centres	One operational picture instead of a wall of disconnected feeds	Unified dashboard connecting camera feeds, trajectories, alerts, heatmaps and prediction outputs
Urban planners	Real traffic-demand data for signal optimisation, parking policy and infrastructure decisions	Aggregated, anonymised mobility patterns that no single camera feed can reveal
Emergency response teams	Situational awareness of how conditions are changing along a route	Live traffic movement, congestion insights and route-condition visibility

5.1 Roles inside the product

Role	Permissions	Denied
officer	Search plates, view trajectories, view analytics dashboard, view and receive alerts	Cannot manage the blacklist, cameras or users
admin	Everything an officer can do, plus manage cameras, blacklist entries and users	Still cannot take automated enforcement action — none exists

5.2 Primary user journey

Officer logs in → types a partial plate into search → autocomplete suggests known plates → selects a plate → map draws every sighting in time order → timeline shows camera, time, confidence → switches to analytics for density and hotspots → live alert arrives on the socket if the plate is blacklisted

6. Product Scope: MVP, Phase 2 and Future

UrbanPulse AI is a large product vision delivered in disciplined stages. The full vision — multi-modal appearance re-identification, a full city road graph, probabilistic route reconstruction and learned next-location prediction — is the product this PRD describes. The **hackathon MVP is a deliberate, defensible subset** of it. Building the whole vision at once would leave nothing finished; building the subset leaves a working end-to-end system with an honest, well-argued roadmap behind it.

Scope discipline principle: *rules that work are a better MVP than models that need data we do not have.* Every capability below is placed in a stage based on whether the data and time to build it credibly exist right now.

6.1 Scope allocation

Capability	Stage	Rationale
Vehicle and plate detection (YOLO)	MVP	Model already fine-tuned and performing well; no further tuning time allocated here.
Multi-object tracking (ByteTrack)	MVP	Required for multi-frame OCR voting to group frames of the same physical vehicle.
Plate OCR (PaddleOCR, fine-tuned)	MVP	Documented multi-line plate support — essential for stacked two-wheeler and auto plates.
Indian plate grammar / RTO correction	MVP	Cheap, high-value accuracy layer over raw OCR output.
Multi-frame OCR voting	MVP	The core differentiator; depends only on tracking, which is in scope.
Geo-tagged sighting storage (PostGIS)	MVP	Foundation for every downstream feature.
Plate-based cross-camera trajectory	MVP	Identity by plate match is simpler and sufficient for the demo scenario.
Blacklist matching and live alerts	MVP	Highest-visibility capability for evaluators; simple to implement reliably.
Rule-based anomaly detection	MVP	Works from day one with no labelled anomaly data.
Analytics: heatmap and summary	MVP	Direct query over stored sightings; PostGIS spatial index makes it fast.
GIS dashboard (Leaflet + kepler.gl)	MVP	Two screens, two purpose-built tools; buildable against mock data from day one.
Auth, roles and audit	MVP	Required for the privacy story to be credible, not optional polish.
Isolation Forest anomaly scoring	Phase 2	Unsupervised, so no labels needed — but needs enough accumulated sighting data to train on.
City mobility graph (OSMnx + NetworkX)	Phase 2	Requires real camera GPS coordinates and a validated OSM zone extract.
Probabilistic route reconstruction	Phase 2	Depends on the mobility graph existing first.
Next-location prediction (Markov → LightGBM)	Phase 2	Needs historical transition data that does not exist yet.
Visual vehicle re-ID (FastReID / OSNet)	Future	A second heavy model plus an embedding database — real engineering weeks. Honest answer to "what if the plate is unreadable".
ML identity fusion (XGBoost)	Future	Exists only to fuse plate confidence with visual re-ID confidence; nothing to fuse until re-ID exists.
Sequence models (LSTM / GRU / temporal GNN)	Future	Research-grade; requires far more trajectory data than is available.
Edge inference at the camera	Future	Production deployment concern; a bandwidth and privacy benefit at city scale.

6.2 Explicitly out of scope

- Face recognition, person identification or any biometric processing.
- Automated penalties, challans or any enforcement action taken without a human.
- Live production RTSP ingestion from municipal cameras during the hackathon — a simulator stands in.
- Mobile applications; the deliverable is a web dashboard.

7. Solution Overview and Core Differentiators

7.1 The nine-stage solution

#	Stage	What happens
1	Detect vehicles and plates	YOLO locates vehicles and number plates in each sampled frame of the CCTV feed.
2	Track across frames	ByteTrack assigns a stable track ID so many frames are known to be one physical vehicle.
3	Read and refine the plate	OpenCV preprocessing (crop, deskew, CLAHE contrast) then fine-tuned PaddleOCR, with multi-frame voting across the track.
4	Correct noisy reads	Indian plate grammar, valid RTO state codes and common character confusions (8/B, O/O, 1/I, 5/S) constrain the output.
5	Store the observation	Plate, camera, timestamp, latitude, longitude and confidence written to PostgreSQL + PostGIS.
6	Connect across cameras	Sightings of the same plate at different cameras are linked into one ordered journey.
7	Reconstruct the unobserved route	Candidate paths between two sightings are generated over the road graph and ranked by plausibility.
8	Predict and flag	Ranked next-camera prediction, plus rule and Isolation Forest anomaly scoring.
9	Visualise and alert	Trajectories, heatmaps and live alerts delivered to the GIS dashboard.

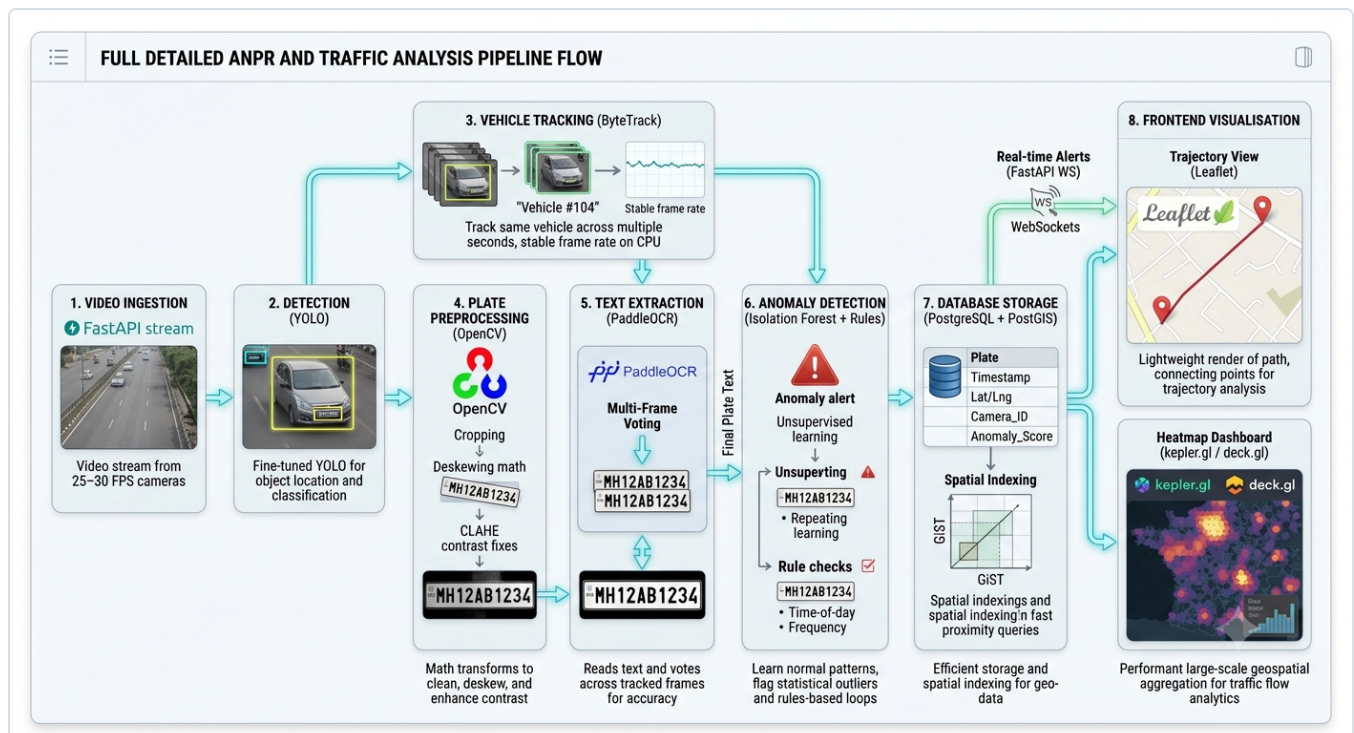


Figure 1 — End-to-end ANPR and traffic analysis pipeline: ingestion, detection, tracking, preprocessing, text extraction, anomaly detection, storage and frontend visualisation.

7.2 Innovation and uniqueness

01 · Trajectory-conditioned OCR

Uses multiple frames and repeated sightings of the same vehicle to correct noisy plate reads, instead of treating every OCR output independently. Frame consensus plus journey evidence produces a more reliable vehicle identity than any single read.

02 · Indian plate-aware decoding

Constrains OCR output using valid Indian plate grammar, RTO state codes and common character confusions. This is context-aware correction applied during decoding, not just a post-processing regex bolted on afterwards.

03 · From ANPR to traffic intelligence

Transforms plate detections into searchable trajectories, anomaly alerts and geospatial traffic analytics using the city's existing camera network. One pipeline: observe → identify → connect → analyse.

7.3 Five novel concepts in the full product vision

#	Concept	Definition
N-01	Multi-modal vehicle identity	Instead of trusting one plate reading, the system weighs several independent pieces of evidence — OCR text with its own confidence, appearance, colour, vehicle type, and spatial-temporal feasibility — into a single confidence estimate. It never asserts a match as certain.
N-02	Probabilistic reconstruction of unseen journeys	Between two sightings the system generates candidate routes over the road network and ranks them by plausibility. No false certainty — it ranks plausible possibilities.
N-03	The city as a mobility graph	Cameras and junctions are nodes; every edge stores road distance, minimum / typical / maximum travel time and historical route share, so a route between two sightings is a physical possibility rather than a straight line on a map.
N-04	Predictive next-location intelligence	Given the sightings so far, the current location, time of day and the mobility graph, the system ranks where the vehicle is likely to be observed next — guidance for an authorised operator deciding where to look, never an automated action.
N-05	Uncertainty-aware decision support	Every match, route, prediction and anomaly carries a confidence value. An anomaly marks movement an authorised human should look at; it is never evidence of wrongdoing.

8. System Architecture

8.1 Three logical layers

1 • Perception layer

CCTV / ANPR streams → video ingestion (OpenCV, RTSP, FFmpeg) → vehicle and plate detection (YOLO) → tracking and OCR (ByteTrack, PaddleOCR).

Output: structured observation events.

2 • Fusion & storage layer

Identity resolution with confidence scoring, then persistence into the event and geo database (PostgreSQL + PostGIS).

Output: scored, geo-tagged vehicle identity records.

3 • Intelligence layer

Mobility graph → route reconstruction (k-shortest paths and scoring) → prediction and anomaly (Markov / LightGBM / Isolation Forest) → FastAPI and GIS dashboard.

Output: predictions, anomaly scores, visual intelligence.

8.2 Deployment architecture for the prototype

The prototype is deliberately split into **two backend services plus one frontend**, not four microservices. This keeps the moving parts within a hackathon timeline while preserving clean service boundaries.

Component	Port	Purpose
Service A — AI Inference Service	8001	Takes an image, returns a plate number with confidence and bounding box.
Service B — Main Backend	8000	Database, ingestion, trajectory search, analytics, auth, blacklist, alerts. One codebase, separate route files per owner.
Frontend	5173	React GIS dashboard. Talks only to Service B, never directly to Service A.
Camera Simulator	—	Stands in for real cameras: reads sample video/images, calls Service A, forwards results to Service B's ingest endpoint.

[Simulator] → Service A /read-plate → [Simulator] → Service B /ingest → Database
[Frontend] → Service B only

Why the frontend never calls Service A. The AI service only matters when a camera sends a new frame. Because there are no real cameras in the prototype, the simulator plays that role. The frontend queries Service B for results exactly as it would with real cameras — so replacing the simulator with live RTSP feeds later requires zero frontend changes.

8.3 Architecture principles

Principle	Implication
Modular and scalable pipeline	CV, backend, database and frontend are independent components; each can be scaled, replaced or tested alone.
Contract-first development	Four interface schemas are frozen up front so all six members build in parallel from day one against fixtures.
Journey-level reliability	Identity decisions are made over a sequence of evidence, never a single frame.

Spatial intelligence in the database	PostGIS handles proximity and region queries natively rather than in application code.
Real-time delivery	FastAPI WebSockets push alerts and live updates; the dashboard never polls.
No GPU dependency for the demo	Models exported to ONNX Runtime so the demo runs on a standard laptop CPU.

9. Functional Requirements

Priority key: **MUST** required for the prototype demo **SHOULD** Phase 2 **COULD** future scope.

9.1 FR-1 — Perception and ANPR

ID	Requirement	Priority	Owner
FR-1.1	The system shall ingest video from a recorded file or RTSP/HTTP stream and sample frames at a fixed rate.	MUST	Perception
FR-1.2	The system shall detect vehicles and number plates in each sampled frame using the fine-tuned YOLO model.	MUST	Perception
FR-1.3	The system shall assign a stable track ID to each vehicle across consecutive frames of one camera using ByteTrack.	MUST	Perception
FR-1.4	The system shall preprocess each plate crop with cropping, deskewing and CLAHE contrast correction before OCR.	MUST	Perception
FR-1.5	The system shall read plate characters using a PaddleOCR recognition model fine-tuned on Indian plate crops, including stacked two-row plates.	MUST	Perception
FR-1.6	The system shall perform multi-frame voting across all reads belonging to one track and emit the consensus plate string.	MUST	Perception
FR-1.7	The system shall validate and correct the consensus string against Indian plate grammar, valid RTO state codes and known character confusions.	MUST	Perception
FR-1.8	The system shall return a confidence value between 0.0 and 1.0 with every plate read, and shall report a machine-readable reason when no plate is returned.	MUST	Perception
FR-1.9	The system should degrade gracefully in night, rain, fog and glare conditions using preprocessing rather than failing silently.	SHOULD	Perception

9.2 FR-2 — Vehicle identity and cross-camera linkage

ID	Requirement	Priority	Owner
FR-2.1	The system shall treat plate text as the primary identity signal and match on exact string comparison first, then fuzzy comparison.	MUST	Identity
FR-2.2	The system shall group all sightings sharing a plate number into a single searchable vehicle journey.	MUST	Identity
FR-2.3	The system shall gate any cross-camera match by a time-plausibility window so physically impossible links are rejected.	MUST	Identity
FR-2.4	The system should extract appearance embeddings from vehicle crops and fall back to cosine similarity when the plate is unreadable.	COULD	Identity
FR-2.5	The system should fuse plate, appearance, colour, vehicle type, time feasibility and geographic feasibility into one explainable weighted identity score.	COULD	Identity

FR-2.6	The system shall band every identity confidence as HIGH (> 0.85), MEDIUM (0.60–0.85) or LOW (< 0.60) for operator display.	SHOULD	Identity
--------	--	--------	----------

9.3 FR-3 — Mobility graph and trajectory reconstruction

ID	Requirement	Priority	Owner
FR-3.1	The system shall return the full sighting history of a plate sorted oldest to newest, ready to draw as a map path without client-side re-sorting.	MUST	Backend
FR-3.2	The system should build a road network graph for one target city zone from OpenStreetMap and snap each camera to its nearest road edge.	SHOULD	Mobility
FR-3.3	The system should generate k-shortest candidate paths between two consecutive sightings of the same vehicle.	SHOULD	Mobility
FR-3.4	The system should score each candidate path on route distance, available time, expected travel time, traffic conditions and historical route share, then output a probabilistic ranking.	SHOULD	Mobility
FR-3.5	Each graph edge shall store road distance, minimum / typical / maximum travel time and historical movement probability.	SHOULD	Mobility

9.4 FR-4 — Prediction and anomaly detection

ID	Requirement	Priority	Owner
FR-4.1	The system shall flag anomalies using an explainable rules engine covering impossible speed, travel-time anomaly, route deviation, odd-hour movement and loitering or repeated loops.	MUST	Intelligence
FR-4.2	Every anomaly alert shall carry a human-readable list of reasons; a score alone is not an acceptable output.	MUST	Intelligence
FR-4.3	The system should layer an unsupervised Isolation Forest model over trajectory features and combine it with the rule score into a final anomaly score.	SHOULD	Intelligence
FR-4.4	The system should predict the next likely camera as a ranked Top-K list with calibrated probabilities, never as a single certain answer.	SHOULD	Intelligence
FR-4.5	Prediction shall use the mobility graph to remove physically unreachable candidate cameras before the model ranks the remainder.	SHOULD	Intelligence
FR-4.6	The system could upgrade prediction from a Markov transition baseline to LightGBM, and later to sequence or graph neural models.	COULD	Intelligence

9.5 FR-5 — Backend, ingestion and analytics

ID	Requirement	Priority	Owner
FR-5.1	The system shall expose an ingestion endpoint that accepts a plate reading with confidence, camera ID and timestamp, and persists it as a sighting.	MUST	Backend
FR-5.2			Backend

	Ingestion shall be authenticated by an internal service API key, distinct from the user login token.	MUST	
FR-5.3	Ingestion shall automatically check the blacklist, create an alert and push it over the WebSocket when a match occurs, with no extra call from the caller.	MUST	Backend
FR-5.4	The system shall provide autocomplete-style search over previously seen plates as the operator types.	MUST	Backend
FR-5.5	The system shall return heatmap points as latitude, longitude and a weight representing sighting count near that point in a time window.	MUST	Backend
FR-5.6	The system shall return a summary of total vehicles tracked, busiest camera, average speed and congestion hotspots with a severity band.	MUST	Backend
FR-5.7	The system shall support camera registration and listing, with creation restricted to the admin role.	MUST	Backend

9.6 FR-6 — Authentication, blacklist and alerts

ID	Requirement	Priority	Owner
FR-6.1	The system shall authenticate users with a username and password and issue a bearer token with a stated expiry.	MUST	Auth
FR-6.2	The system shall enforce two roles — admin and officer — with blacklist, camera and user management restricted to admin.	MUST	Auth
FR-6.3	The system shall allow an admin to add, list and remove blacklist entries, each recording a reason, who added it and when.	MUST	Auth
FR-6.4	The system shall maintain a historical alert log, paginated, with alert type and severity on every entry.	MUST	Auth
FR-6.5	The system shall push new alerts to authenticated clients over a WebSocket held open after login, with no polling.	MUST	Auth
FR-6.6	The system shall record an audit log entry for every query and every blacklist modification.	SHOULD	Auth

9.7 FR-7 — Frontend and GIS dashboard

ID	Requirement	Priority	Owner
FR-7.1	The dashboard shall provide a login screen and hold the session token for all subsequent calls.	MUST	Frontend
FR-7.2	The dashboard shall provide a plate search bar with live autocomplete, opening the selected plate's trajectory.	MUST	Frontend
FR-7.3	The dashboard shall render a vehicle's journey as ordered map markers connected by a polyline, using Leaflet.	MUST	Frontend
FR-7.4	The dashboard shall render an observation timeline showing camera name, timestamp and confidence for each sighting.	MUST	Frontend
FR-7.5	The dashboard shall render the analytics heatmap using kepler.gl / deck.gl fed directly by the heatmap endpoint's weighted points.	MUST	Frontend

FR-7.6	The dashboard shall show an alerts panel: historical alerts on load, then live alerts appended from the WebSocket.	MUST	Frontend
FR-7.7	The dashboard shall visually encode confidence (colour or opacity) so low-confidence evidence is never presented as certain.	MUST	Frontend
FR-7.8	The dashboard shall provide admin screens for blacklist and camera management, hidden from officer-role users.	MUST	Frontend
FR-7.9	The dashboard shall be buildable against mock JSON fixtures matching the locked contract, with no dependency on backend readiness.	MUST	Frontend

10. Technology Stack (Locked)

Every tool below was selected against the actual system being built, not copied from a reference list. Where a category is genuinely useful it is kept and justified; where it is scope creep beyond the MVP it is marked as a later stage rather than quietly included.

10.1 Consolidated stack

Category	Selected technology	Version	Stage	Why this choice
Core language	Python	3.11	MVP	Every other tool in the stack is Python-native.
Computer vision	OpenCV	—	MVP	Preprocessing: crop, deskew, CLAHE contrast.
Deep learning engine	PyTorch (via Ultralytics)	2.12.1	MVP	Underlying engine for YOLO; called through Ultralytics.
Vehicle & plate detection	YOLO (Ultralytics, fine-tuned)	v8 / v11	MVP	Already fine-tuned and performing well; detection is the solved stage.
Multi-object tracking	ByteTrack	—	MVP	Same accuracy as BoT-SORT on dense traffic but faster and more stable on CPU. BoT-SORT's advantage is camera-motion compensation, which is irrelevant for fixed junction cameras.
OCR / recognition	PaddleOCR (recognition module fine-tuned)	—	MVP	Documented multi-line plate support — critical for stacked two-wheeler and auto plates common in Indian traffic.
Classical ML	scikit-learn	1.9.0	MVP	Isolation Forest, clustering and calibration, no extra install.
Anomaly detection	Rules engine → Isolation Forest	—	MVP / P2	Unsupervised — no labelled anomaly set needed, which the team does not have and cannot build in time.
Database	PostgreSQL	18.6	MVP	Relational store for sightings, cameras, users, alerts.
Spatial extension	PostGIS	3.6.2	MVP	GiST spatial indexing makes proximity queries for the heatmap fast at tens of thousands of rows.
Backend framework	FastAPI	0.136.3	MVP	Async-native with first-class WebSocket support, required for live alerts.
Validation	Pydantic	2.x	MVP	Enforces the locked contract shapes at the boundary.

ORM	SQLAlchemy	2.0.52	MVP	Standard data-access layer over PostgreSQL.
Frontend	React + Vite	—	MVP	Component model suits the dashboard; kepler.gl embeds as a React component.
Trajectory map view	Leaflet	—	MVP	Lightweight, simple markers and polylines — exactly the trajectory screen, nothing more.
Analytics heatmap	kepler.gl (on deck.gl)	—	MVP	Purpose-built for large geolocation datasets and in-browser spatial aggregation; consumes the heatmap endpoint's weighted points directly.
Inference runtime	ONNX Runtime	—	MVP	Runs on CPU, GPU and mobile without lock-in. TensorRT would require an NVIDIA GPU and an on-site engine rebuild — an unacceptable demo-day risk.
Deployment	Docker + docker-compose	—	MVP	One command brings up Service A, Service B and the database, independent of any laptop's Python version.
Road graph	OSMnx + NetworkX	2.1.1 / 3.x	P2	Turns OpenStreetMap into a routable graph; needed for reconstruction and prediction.
Tabular prediction	LightGBM	4.6.0	P2	Fast and strong on structured trajectory features; easier than deep sequence models initially.
Alternative boosting	XGBoost	3.4.1	P2	Benchmark and backup for tabular prediction and identity fusion.
Prediction baseline	Markov transition model	—	P2	Simple, explainable camera-to-camera transition probabilities before any complex ML.
Vehicle re-ID	FastReID / OSNet	—	Future	Needed only when the plate cannot be read at all; a second heavy model plus an embedding database.
Cache / queue	Redis	—	Future	Optional at prototype scale; useful for live feed buffering in production.

10.2 Deliberate exclusions

Excluded	Reason
BoT-SORT	

	Its one advantage over ByteTrack is camera-motion compensation, which does not apply to fixed traffic cameras. ByteTrack is faster at equivalent accuracy.
MapLibre	A third overlapping map renderer. Leaflet covers the trajectory screen and kepler.gl covers the heatmap; nothing is left for it to do.
TensorRT	Requires an NVIDIA GPU and a GPU-specific engine build. The demo must run on a standard laptop CPU.
ML identity fusion in the MVP	It exists to fuse plate confidence with visual re-ID confidence. Without re-ID there is nothing to fuse.
Neo4j graph database	NetworkX in-process is sufficient at prototype graph size; a dedicated graph DB is a future scaling decision.

11. Data Model and Database Design

11.1 Core data model flow

CAMERA → OBSERVATION → VEHICLE IDENTITY CANDIDATE → IDENTITY CONFIDENCE
→ TRAJECTORY → PREDICTION / ANOMALY

11.2 Core tables

Table group	Tables	Holds
Camera registry	cameras , camera_locations	Camera ID, human-readable name, latitude, longitude, road snap reference.
Observations	vehicle_observations , vehicle_tracks , plate_observations	Every sighting: plate, camera, timestamp, geo point, confidence, track reference, bounding box.
Identity	vehicle_identity_clusters	Grouped sightings believed to be one physical vehicle, with a fusion confidence.
Road network	road_edges , camera_connections	Graph edges with distance, travel-time bands and historical movement probability.
Trajectories	trajectory_candidates	Candidate reconstructed paths between two sightings, each with a probability.
Intelligence	predictions , anomaly_events	Ranked next-camera predictions and scored anomaly events with reasons.
Operations	users , blacklist , alerts , audit_log	Accounts and roles, watchlist entries, alert history, query audit trail.

11.3 What PostGIS is used for specifically

- **Geographic coordinates** — every sighting stores a real spatial point, not two loose float columns.
- **Spatial queries** — "how many sightings near this point", "which cameras lie in this zone".
- **Distance calculations** — camera-to-camera and route-segment distances.
- **Map visualisation** — aggregation feeding the heatmap endpoint directly.

GiST spatial indexing keeps proximity aggregation fast even as sightings grow into the tens of thousands, which is exactly the query shape the analytics heatmap requires.

11.4 Global data rules

Rule	Specification
Transport format	JSON everywhere, except image upload which is multipart/form-data .
Timestamps	ISO 8601 UTC strings, e.g. 2026-08-27T10:15:32Z .
Plate numbers	Stored and transmitted uppercase with no spaces, e.g. MH12AB1234 .
Coordinates	Plain floats: "lat": 18.5204, "lng": 73.8567 .
Confidence	Float between 0.0 and 1.0 , inclusive.
Sort order	Sightings are always returned oldest → newest; the client never re-sorts.

Error shape	{ "error": true, "message": "...", "code": "MACHINE_READABLE_CODE" } on every service and endpoint.
Status codes	200 OK · 201 Created · 400 validation failed · 401 missing/invalid token · 403 forbidden for this role · 404 not found · 500 server error.

12. API Contract (Locked Spec v1.0)

This section is the single source of truth for all interfaces. Nobody changes a field name or type here without notifying every consumer of that endpoint and getting explicit agreement first.

12.1 Service A — AI Inference Service (:8001)

Method	Path	Behaviour
GET	/health	Liveness and model version. Called by Service B on startup and by the simulator before it begins sending frames.
POST	/api/v1/read-plate	Core AI endpoint. Accepts one image (and optional camera_id for logging), returns plate number, confidence, bounding box, raw OCR text, state-code validity and processing time.

Failure semantics: a frame with no readable plate is **not** an error — it returns 200 with success: false and a machine-readable reason of NO_PLATE_DETECTED, LOW_CONFIDENCE or INVALID_FORMAT. 400 is reserved for a corrupt image and 500 for a model crash.

12.2 Service B — Main Backend (:8000)

Method	Path	Access	Purpose
POST	/api/v1/auth/login	Public	Returns a bearer token, its expiry and the user's role.
GET	/api/v1/auth/me	Token	Returns the current username and role.
POST	/api/v1/ingest	API key	Called by the simulator. Saves a sighting, returns a sighting ID and whether an alert was triggered.
GET	/api/v1/trajectory/{plate}	Token	Full ordered sighting history for a plate; optional from_date / to_date.
GET	/api/v1/plates/search	Token	Autocomplete over previously seen plates.
GET	/api/v1/cameras	Token	List all registered cameras with coordinates.
POST	/api/v1/cameras	Admin	Register a new camera.
GET	/api/v1/analytics/heatmap	Token	Weighted points for the density surface.
GET	/api/v1/analytics/summary	Token	Totals, busiest camera, average speed, congestion hotspots.
GET	/api/v1/blacklist	Token	List watchlist entries with reason and provenance.
POST	/api/v1/blacklist	Admin	Add a watchlist entry.
DELETE	/api/v1/blacklist/{plate}	Admin	Remove a watchlist entry.
GET	/api/v1/alerts	Token	Paginated historical alert log.
WS	/ws/alerts?token=	Token	

			Held open after login; the server pushes each new alert automatically, no polling.
--	--	--	--

12.3 Controlled vocabularies

Field	Permitted values
reason (Service A)	NO_PLATE_DETECTED , LOW_CONFIDENCE , INVALID_FORMAT
alert_type	BLACKLIST_MATCH , ANOMALY
severity (alerts)	critical , warning , info
severity (congestion)	low , medium , high
role	admin , officer

12.4 Screen-to-endpoint map

Frontend screen	Endpoints called
Login page	POST /auth/login
Plate search bar	GET /plates/search then GET /trajectory/{plate}
Map / trajectory view	GET /trajectory/{plate} , GET /cameras
Analytics dashboard	GET /analytics/heatmap , GET /analytics/summary
Alerts panel	GET /alerts on load, then WS /ws/alerts
Admin: blacklist	GET/POST/DELETE /blacklist
Admin: cameras	GET/POST /cameras

12.5 Intelligence-layer contracts (Phase 2)

The intelligence module exposes its own endpoints, returning ranked predictions and explainable anomaly objects:

Endpoint	Returns
GET /vehicles/{vehicle_id}	Identity record for one vehicle.
GET /vehicles/{vehicle_id}/trajectory	Reconstructed trajectory with per-hop probability.
GET /vehicles/{vehicle_id}/prediction	Top-K next cameras with calibrated probabilities.
GET /vehicles/{vehicle_id}/anomalies	Anomaly score, severity band and the list of reasons.
POST /prediction/next-location	On-demand prediction for a supplied observation sequence.
POST /anomaly/check	On-demand anomaly evaluation for a supplied trajectory.

12.6 Working agreements

- **The frontend never waits.** Copy the exact JSON shapes into mock fixture files and build every screen from day one; swapping to a real `fetch()` later requires no shape change.
- **Postman before "done".** Every endpoint is verified in a shared Postman collection, exported into the repository's `/docs` folder, before it is declared complete.
- **Contract changes are announced.** Post the proposed change, get agreement from every consumer, then update this document and bump its version.

13. AI / ML Specification

The intelligence layer sits **after** identity, trajectory reconstruction and the mobility graph. Its processing chain is:

```
Trajectory → Next-location prediction → Anomaly detection → Confidence / intelligence scoring  
→ API
```

Not every model is built from scratch. Perception models (YOLO for detection, PaddleOCR for OCR, and re-ID backbones if adopted later) are existing pretrained components. What is built here is the reasoning that consumes their outputs.

13.1 Identity fusion scoring

The MVP approach is an **explainable weighted scoring system**, chosen deliberately over ML fusion so that every identity decision can be debugged and defended:

$$\text{Score} = w_1(\text{Plate}) + w_2(\text{Re-ID}) + w_3(\text{Colour}) + w_4(\text{Time}) + w_5(\text{Location})$$

Confidence band	Range	Operator meaning
HIGH	> 0.85	Strong evidence; safe to present as a likely match.
MEDIUM	0.60 - 0.85	Plausible; presented with visible uncertainty.
LOW	< 0.60	Weak evidence; shown only as a candidate for human review.

Only once the explainable version is working and debuggable does the team experiment with non-linear ML fusion (XGBoost, LightGBM or logistic regression) as an advanced upgrade.

13.2 Next-Location Prediction System (NLPS)

Goal: given a vehicle's latest observations, predict its next likely camera. The system provides **ranked** next-location predictions rather than absolute certainty.

```
Vehicle identity → Trajectory history → Current camera → Mobility graph  
→ Generate reachable cameras → Remove impossible candidates → Feature engineering  
→ LightGBM prediction → Probability calibration → Top-K next cameras
```

The graph does the elimination and the model does the ranking. LightGBM is selected over LSTM or Transformer models initially because the inputs are largely structured and tabular; the target is `next_camera_id`. A Markov transition baseline ships first because it is simple, explainable and sufficient to demonstrate the concept.

NLPS feature set

Sequence context	Temporal	Graph and route
<code>current_camera</code> <code>previous_camera</code> <code>previous_2_camera</code>	<code>hour</code> <code>day_of_week</code> <code>time_gap</code> <code>expected_travel_time</code> <code>observed_travel_time</code>	<code>route_distance</code> <code>route_probability</code> <code>road_class</code> <code>candidate_camera_distance</code>

candidate_path_probability
historical_transition_frequency

13.3 Anomaly detection

Two layers, deliberately in this order. The rules engine is the MVP because it detects impossible speed and route deviation immediately without requiring any training data. Isolation Forest is the ML upgrade because it is unsupervised — there is no labelled set of anomalous driving, and one cannot realistically be built inside a hackathon.

Vehicle trajectory → Feature extraction → Rule engine → Isolation Forest
→ Combined anomaly score → Explainable alert

Rule checks (v1)	ML feature set (v2, Isolation Forest)
Impossible speed Travel-time anomaly Route deviation Odd-hour movement Loitering and repeated loops Rare camera sequence	speed, acceleration, travel_time, expected_travel_time, speed_ratio, route_deviation, route_probability, time_of_day, day_of_week, dwell_time, loop_count, camera_transition_frequency

$$\text{FinalAnomaly} = (\text{RuleScore} \times \text{RuleWeight}) + (\text{MLScore} \times \text{MLWeight})$$

Standing instruction: do not blindly trust the ML model. The rule-based explanation is always retained alongside the score, so every alert can tell an operator *why* it fired. A score with no reason is not a shippable output.

13.4 Evaluation methodology

Aspect	Specification
Primary metrics	Top-1 accuracy, Top-3 accuracy, Top-5 accuracy, Mean Reciprocal Rank, log loss, calibration error.
Split rule	Chronological, never random. For example: January–August train, September validation, October test.
Why chronological	A random split leaks future movement patterns into training and produces an accuracy figure that will not survive contact with real deployment.
Reporting rule	Accuracy depends heavily on camera coverage, data quality and dataset size. No headline accuracy figure is promised in advance of measurement.

14. Datasets and Data Strategy

14.1 Datasets in use

Dataset	Serves	Role
HuggingFace — Indian Number Plates	Perception / OCR	Indian plate imagery for detection and recognition fine-tuning.
Kaggle — Indian License Plates with Labels	Perception / OCR	Labelled plate crops for the recognition module.

Roboflow ILPD — Indian Licence Plate Detection	Perception	Detection training and validation set.
SkyEye	Trajectory / prediction	Primary Indian trajectory behaviour source — aerial mixed-traffic, 4,021 tracks.
DRISHT-E	Trajectory / prediction	Secondary Indian traffic movement and trajectory source.
ITD	Perception support	Indian traffic and perception support data.
CityFlowV2	Multi-camera validation	Primary benchmark for multi-camera vehicle tracking and re-ID; validates the algorithmic pipeline.
highD / BDD100K	Experimentation only	Generic trajectory and temporal experimentation — explicitly not for Indian domain adaptation.

14.2 The critical data gap

Identified gap: there is no confirmed public Indian dataset providing exactly what UrbanPulse needs — **synchronised, multi-camera sequences of the same vehicle across disconnected CCTV nodes at scale**. This is the single largest constraint on the intelligence layer, and it is stated openly rather than papered over.

Action plan

1. Use public Indian datasets to learn **movement behaviour** characteristic of Indian traffic.
2. Use CityFlowV2 to **validate the algorithmic pipeline** for multi-camera logic.
3. Create or obtain an **UrbanPulse-specific Indian camera-sequence dataset** for final training.
4. Until that exists, the prototype demonstrates the pipeline on a controlled scripted scenario and reports its limits honestly.

14.3 Data still required before model delivery

- Actual Indian multi-camera trajectory data and target-city camera coordinates.
- Historical camera-to-camera transition data for the Markov baseline and LightGBM training.
- Executed chronological training / evaluation split.
- A trained LightGBM model with measured anomaly precision and recall.

15. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance — CV	The detection and tracking pipeline shall sustain a usable, stable frame rate on the demo hardware (standard laptop CPU) with models exported to ONNX Runtime.
NFR-02	Performance — API	Trajectory and analytics queries shall return fast enough for interactive dashboard use at prototype data volume, backed by PostGIS GiST spatial indexing.
NFR-03	Latency — alerts	A blacklist match shall appear on the dashboard within seconds of ingestion, delivered by WebSocket push rather than polling.
NFR-04	Scalability	The pipeline shall be horizontally scalable across additional cameras and zones; the perception layer is designed to move to the edge in production.
NFR-05	Portability	The demo shall not depend on any specific GPU being present. No inference path may require an on-site GPU-specific engine rebuild.
NFR-06	Reproducibility	A single <code>docker compose up</code> shall bring Service A, Service B and the database live on a clean machine.
NFR-07	Maintainability	Modular components with frozen interface contracts; no module may reach into another's internals.
NFR-08	Testability	Every interface contract shall have sample fixtures; every endpoint shall be exercised in the shared Postman collection before being declared done.
NFR-09	Security	Token-authenticated API, role-based authorisation, a separate internal API key for service-to-service ingestion, and no credentials in source control.
NFR-10	Data protection	Encryption in transit and at rest, defined retention periods, and aggregation for any planning-oriented use.
NFR-11	Auditability	Every query and every watchlist change shall be attributable to a user and a time.
NFR-12	Explainability	No score is shipped without the reasons behind it; identity, prediction and anomaly outputs are all human-interpretable.
NFR-13	Economic	The stack is entirely open source, substantially reducing cost against proprietary ANPR and analytics platforms.

16. Privacy, Ethics and Responsible Deployment

Foundational position. UrbanPulse AI is a **decision-support system, not an autonomous enforcement system**. It narrows where an authorised human should look and shows the confidence behind every suggestion. It never decides, penalises or acts.

Control area	Commitment
Access control	Authorised users only, role-based permissions, and a full audit log of every query.
Data discipline	Encryption in transit and at rest, defined retention periods, and aggregation for planning use so individual records are not the unit of analysis.

Human in the loop	Every match, prediction and anomaly is reviewed by an authorised person before any action is considered.
Anomaly framing	An anomaly is never evidence of wrongdoing. It marks movement that differs from a learned pattern and warrants a look — nothing more.
PII handling	Vehicle plates are treated as personally identifying data and are subject to secure storage, access control and retention policy.
Scope limitation	No face recognition, no person identification, no biometric processing anywhere in the system.
Edge processing benefit	Moving perception to the camera in production keeps raw video local, reducing both bandwidth and the volume of stored imagery — a privacy benefit as much as an engineering one.
Uncertainty disclosure	Confidence is displayed, never hidden. A low-confidence read is presented as uncertain evidence, not as a fact.

17. Feasibility, Risks and Mitigations

17.1 Feasibility assessment

Dimension	Assessment
Technical	Built entirely on a proven open-source stack: OpenCV, YOLO, ByteTrack, PaddleOCR, PostgreSQL + PostGIS, FastAPI. No unproven component sits on the critical path.
Data	Works with standard RTSP CCTV feeds. Public Indian plate datasets exist for perception; the multi-camera sequence gap is documented and planned around.
Hardware	The demo runs on a standard laptop CPU via ONNX Runtime. A GPU-enabled server improves throughput in production but is not required to demonstrate the system.
Operational	Modular architecture with containerised deployment makes setup, monitoring and maintenance straightforward.
Economic	The open-source stack significantly reduces cost against proprietary ANPR and analytics systems, and adds value to camera infrastructure the city already owns.
Scalability	Horizontally scalable pipeline supporting more cameras, more zones and long-term data growth.

17.2 Risk register

ID	Risk	Impact	Mitigation
R-01	Environmental variation — night, rain, fog, glare	Reduced detection and OCR accuracy	Advanced preprocessing (CLAHE, dehaze, denoise), diverse training data, and multi-frame voting so one bad frame does not decide identity.
R-02	Occlusion and heavy crowding	Partial plate visibility breaks tracking and identity consistency	ByteTrack association across frames plus OCR voting; low-confidence reads carried as uncertain evidence rather than discarded or trusted.
R-03	Cross-camera matching errors	Similar vehicles or plates incorrectly linked	Time-plausibility gating, Indian plate grammar constraints, and confidence banding so uncertain links are visibly uncertain.
R-04	No Indian multi-camera sequence dataset	Prediction model cannot be trained on domain-realistic data	Rules and graph statistics ship first; CityFlowV2 validates the pipeline; a project-specific dataset is scoped as required work.
R-05	Data quality and volume growth	Query and index performance degrades on large spatial-temporal data	PostGIS spatial indexing, partitioning and archival policy.
R-06	Real-time compute constraints	High-FPS multi-camera input demands significant compute	Fixed-rate frame sampling, ONNX Runtime optimisation, and an edge-inference architecture path for production.
R-07	Demo-day hardware uncertainty	Demo fails if it depends on a specific GPU	

			TensorRT excluded; ONNX Runtime on CPU is the committed inference path; full stack containerised.
R-08	Privacy and compliance	Plate data is PII and mishandling carries real consequences	Role-based access, encryption, audit logs, retention policy, and an explicit no-automated-action design stance.
R-09	Scope creep into research territory	Half-built advanced features and nothing finished	Staged scope in Section 6; advanced capabilities are stated as roadmap, not claimed as built.
R-10	Contract drift between modules	Integration failure late in the build	Four schemas frozen in week one; all six members build against shared fixtures; changes require consumer sign-off.

17.3 Stated limitations

- Accuracy is a function of camera coverage, image quality and dataset size — no universal accuracy figure is claimed.
- Prediction is probabilistic guidance for a human operator, never a determination of where a vehicle will be.
- Without visual re-identification in the MVP, a wholly unreadable plate breaks the identity chain for that sighting. This is disclosed rather than hidden, and re-ID is the named remedy.
- The prototype operates on one city zone and a limited camera set.

18. Team, Ownership and Interface Contracts

18.1 Module ownership

#	Owner	Module	Scope
M1	Rishabh Joshi	Computer Vision / Perception	Video ingestion, vehicle detection, multi-object tracking, plate detection, OCR and text post-processing.
M2	Soham Kale	Vehicle Identity / Re-ID	Cross-camera matching, plate and appearance fusion, identity confidence scoring, global vehicle ID assignment.
M3	Ishwari Honrao	Mobility Graph / Path Reconstruction	Road network graph, camera-to-road snapping, candidate route generation, trajectory reconstruction.
M4	Mayur Kaithwas + Aditya Kale	Backend + AI / ML (joint)	One owns the backend and data layer (PostGIS schema, ingestion, FastAPI endpoints); the other owns the intelligence layer (prediction, anomaly scoring, confidence calibration). Both meet at the API contract.
M5	Ujjwal Jain	Frontend + Dataset Sourcing	React GIS dashboard, plus sourcing and preparing usable datasets for the whole team.
M6	Shared / floating	Integration, Testing, Docs	Fixtures for every contract, end-to-end testing, Docker setup, documentation, demo script and rehearsal. Unblocks whichever module is stuck.

Note on dataset sourcing: in Phase 0 this is the higher-priority half of the frontend role. The other owners cannot validate their approach without sample footage, so the dataset report — source, size, licence, and which module it serves — is a blocking deliverable, not a background task.

18.2 Frozen interface contracts

Boundary	Producer → Consumer	Contract fields
Observation record	M1 → M2	camera_id , timestamp , bbox , crop reference , plate_text , plate_conf , vehicle_class
Identity record	M2 → M3 / M4	global_vehicle_id , member observations , match score , fusion confidence
Trajectory record	M3 → M4	Ordered node/edge path , per-hop probability , time window , gap flags
REST API	M4 → M5	JSON and GeoJSON endpoints for vehicle , trajectory , prediction and anomaly

Parallelism rule. The dependency chain M1 → M2 → M3 → M4 → M5 is a **data** dependency, not a **schedule** dependency. Because all four schemas are frozen up front, every member builds from day one against sample fixtures. The arrows only show whose real output eventually replaces whose fixture.

18.3 Phase-0 submission per owner

One page per module covering: (a) confirmed requirements and open questions, (b) final tech stack with versions, (c) the approach in five to eight steps, and (d) one risk that could make the module unviable, with its fallback. No code required in this phase.

19. Roadmap and Definition of Done

19.1 Strict development priorities

1. Get perception working → 2. Generate structured observation data → 3. Build the mobility graph
→ 4. Build route reconstruction → 5. Add prediction and anomaly intelligence

19.2 Phased rollout — intelligence layer

Step	Task	Exit condition
1	Build the data contracts and feature pipeline	Features computable from stored trajectories.
2	Build the graph-only NLPS baseline	Reachable cameras generated and ranked by transition probability.
3	Train the LightGBM next-camera model	Model trained on a chronological split, metrics recorded.
4	Build the rule anomaly engine	All five rule checks firing with human-readable reasons.
5	Add Isolation Forest	Combined rule + ML score produced.
6	Build confidence calibration	Probabilities calibrated, calibration error measured.
7	Expose through FastAPI	Prediction and anomaly endpoints live and documented.
8	Evaluate on unseen trajectories	Top-K, MRR and log loss reported on the held-out test period.

19.3 MVP priority per module

Module	Build first (prototype)	Add later
M1 Perception	YOLO detection and tracking on recorded video; plate crop and OCR on clear frames; write observations as JSON.	Live RTSP ingestion, night/low-light tuning, multi-camera scaling, plate super-resolution.
M2 Identity	Plate-based matching first — exact then fuzzy — then a pretrained re-ID embedding with cosine matching on two or three cameras.	Fine-tuned re-ID model, learned fusion weights, city-scale clustering, merge/split correction.
M3 Mobility	OSM graph for one city zone; snap cameras to roads; shortest path between two camera hits with a travel-time check.	K-best candidate routes with probabilities, turn restrictions, traffic-aware weights, full-city graph.
M4 Backend / AI	PostGIS schema and FastAPI endpoints over stored observations; rule-based anomalies for speed and deviation.	ML next-location model, learned anomaly scoring, caching, richer live updates.
M5 Frontend	Map with camera markers, vehicle search, plotted route polyline and observation timeline.	Confidence heat styling, playback animation, richer alert panel, filters, responsive polish.
M6 Integration	Sample dataset and fixtures for all four contracts; repo structure; docker-compose for DB and API; smoke test of the full chain.	Annotated evaluation set, CI, performance benchmarks, full docs, rehearsed demo script.

19.4 Prototype definition of done

One vehicle, three cameras, one city zone. Detected and plate-read by M1, linked into a single identity by M2, routed across the road graph by M3, stored and served by M4, and visible as a route with a timeline on M5's map — running end to end inside M6's containers, started with a single command on a machine with no prior setup.

19.5 Beyond the hackathon

Horizon	Focus
Near term	Isolation Forest anomaly scoring, mobility graph on a real city zone, Markov prediction baseline, measured evaluation on collected data.
Medium term	LightGBM prediction with calibrated probabilities, probabilistic route reconstruction with k-best ranking, visual re-identification for unreadable plates.
Long term	Sequence and temporal graph neural models, learned identity fusion, edge inference at the camera with a central intelligence platform, multi-city deployment.

20. Impact and Benefits

20.1 Key impacts

Impact	Description
Stolen / wanted vehicle traceability	City-wide plate search and real-time alerts help locate vehicles faster across multiple camera points.
Unified command visibility	Camera observations, trajectories, alerts and analytics arrive in one operational dashboard instead of separate systems.
Confidence-aware review	Every match, anomaly and prediction carries a confidence value, enabling responsible, human-reviewed action.
Traffic analytics	Heatmaps, vehicle counts and time-based movement trends reveal congestion hotspots and real traffic demand.
Predictive monitoring	The mobility graph plus ranked next-camera prediction directs proactive monitoring rather than reactive search.
Smarter urban planning	Aggregated movement patterns support signal planning, road improvements and data-driven mobility policy.

20.2 Benefit categories

Social Safer roads, fairer enforcement and faster investigation support through searchable trajectories and confidence-based review.	Economic Lower monitoring cost, efficient use of existing infrastructure and improved operational efficiency for city management.	Operational A unified traffic dashboard, city-wide vehicle traceability and predictive decision support for proactive monitoring.	Environmental / urban Supports sustainable mobility, reduces survey dependency and delivers more value without adding new camera infrastructure.
--	---	---	--

20.3 Value proposition in one sentence

No new hardware. The intelligence is added in software over cameras a city already owns — isolated cameras become connected intelligence, and every camera that once saw only a moment now contributes to understanding a journey.

21. Open Questions and Outstanding Items

21.1 Completed in Phase 0

- Final AI/ML architecture and model selection.
- Feature list, NLPS design and anomaly-detection design.
- Input/output contracts and API requirements.
- Dataset strategy, with limitations identified and documented.
- Verified technology stack with MVP scope discipline applied per category.

21.2 Open items

ID	Open question	Owner	Needed by
0-01	Target city and zone boundary, plus real camera GPS coordinates.	Mobility + Frontend	Before the mobility graph can be built
0-02	Does the detector run at usable FPS on the actual demo hardware, and is OCR legible at real camera mounting distance?	Perception	Decides whole-pipeline feasibility
0-03	Does a pretrained re-ID backbone transfer acceptably to Indian traffic footage, or is fine-tuning unavoidable?	Identity	Before any re-ID work is scheduled
0-04	Graph build time and per-query latency — does route ranking run live or offline?	Mobility	Before the route feature is wired to the UI
0-05	Historical camera-to-camera transition data source for the Markov baseline.	Intelligence	Before prediction training
0-06	Map tile provider and licensing terms for the deployed dashboard.	Frontend	Before public deployment
0-07	Reconcile the person-numbering used in the API contract with the module-owner roster so every route has one named owner.	Whole team	Immediately — a naming mismatch here causes integration confusion
0-08	Team ID and registered team name for the SIH portal submission.	Team lead	Before submission
0-09	Retention period and deletion policy for stored sightings.	Backend	Before any real data is ingested

21.3 Standing decisions to preserve

Decision	Rationale to remember
Do not tune YOLO further	Detection is the solved stage of ANPR pipelines. Time spent here buys nothing; time spent on recognition and the reasoning layer buys everything.
ByteTrack over BoT-SORT	

	Equivalent accuracy on dense traffic, faster on CPU, and BoT-SORT's advantage does not apply to fixed cameras.
Explainable scoring before ML fusion	A debuggable weighted score that works beats a learned score nobody can explain to an evaluator or an operator.
Rules before models	Rules that work are a better MVP than models that need data the team does not have.
ONNX Runtime, no GPU dependency	Demo-day hardware is not guaranteed. The inference path must not assume it.
Leaflet and kepler.gl, not one or the other	Two screens with genuinely different jobs: lightweight path rendering, and large-scale spatial aggregation.

22. References

22.1 Key research papers

- **YOLOv8 for License Plate & Character Recognition** — arXiv, 2025.
- **Indian Licence Plate Dataset in the Wild** — Tanwar et al., 2021 (arXiv:2111.06054).
- **How Many Labeled License Plates Are Needed?** — Wu et al., 2018 (arXiv:1808.08410).
- **ByteTrack: Multi-Object Tracking by Associating Every Detection Box** — Zhang et al., ECCV 2022 (arXiv:2110.06864).
- **CityFlow: A City-Scale Benchmark for Multi-Target Multi-Camera Vehicle Tracking & Re-ID** — Tang et al., CVPR 2019.
- **Isolation Forest** — Liu, Ting & Zhou, IEEE ICDM 2008.

22.2 Dataset sources

- HuggingFace — Indian Number Plates Dataset.
- Kaggle — Indian License Plates with Labels.
- Roboflow — ILPD: Indian Licence Plate Detection.
- SkyEye — Indian mixed-traffic trajectory dataset for tracking and behaviour analysis.
- DRISHT-E — Indian drone-based vehicle detection, tracking and trajectory extraction.
- CityFlowV2 — City-scale multi-camera vehicle tracking and re-ID benchmark (AI City Challenge).

22.3 Core technology references

Technology	Reference
PaddleOCR	Official PaddleOCR project documentation
PostGIS	postgis.net
FastAPI	fastapi.tiangolo.com
Leaflet	leafletjs.com
kepler.gl	kepler.gl
ONNX Runtime	onnxruntime.ai
OSMnx / NetworkX	OSMnx and NetworkX project documentation
Ultralytics YOLO	Ultralytics project documentation

Every camera sees a moment. UrbanPulse AI understands the journey.

Team Trace Forge · Smart India Hackathon 2026 · Problem Statement SIH26127 · End of document